

Table Title has the following fields:

featuredImageUrl
featuredImageAuthor
MediaAltText
mediaTitle
mediaCaption

Add Image

When user clicks a button to add an image a popup opens where the user can select using AI, load an image from local machine, loading a stock image or generating an Infographics.

After generating an image and storing it in supabase we will get the image_url. In addition, editable text boxes should be available for metadata for author, AltText, Title, Caption. Metadata fields need to be pre filled automatically if possible and editable by users (e.g. AI generated image can output metadata based on the prompt).

The output from this process should be stored in a new supabase table 'Images' with the following fields (you will need to provide the migration sql to create this table):

user_id
ImageUrl
ImageAuthor
MediaAltText
mediaTitle
mediaCaption

A) Generate AI Image:

The user can enter an image prompt or If the user selected a paragraph, the system it remembers the paragraph, it will call the default LLM and will generate a prompt to generate an image.

The user has the option to choose Stable Disuffussion, Google Imagen or Flux Kontext and nanoBanana. The user is also able to select the desired aspect ratio from a dropdown list.

If the model allows it can generate an image based on an uploaded reference image.

For AI model names and keys, 'llm_providers_image' contain the model description and technical name with fields api_keys_id linking to table 'api_keys' will provide the API key (linked by api_key_id).

Following are the supported models:

name	provider	model_name
Stable Image Ultra (Stable Difussion)	stability.ai	ultra
Imagen 4	Google	imagen-4.0-generate-001
Nano Banana	Google	gemini-2.5-flash-image
Nano Banana 3	Google	gemini-3-pro-image-preview
Stable Image Core (Stable Diffussion)	stability.ai	core
Flux Kontext	fluxapi.ai	kontext

Stable Diffusion 3.5 (Stable Diffussion)	stability.ai	sd3
Imagen 4 Ultra	Google	imagen-4.0-ultra-generate-001
Imagen 4.0 Fast	Google	imagen-4.0-fast-generate-001

The resulting image should be displayed for evaluation and provide the option to change prompt and/or generate a new image. The prompt can be used as a meta description/ media caption with AI as media author, media alt text.

Please review functionality of each model above and identify functionalities available that can improve the image generation (e.g. Nano Banana can improve an input picture or “Flux Kontext” **Object & Text Manipulation or Style Transfer**.

To identify the default llm you need to check table llm_providers, field ‘is_default’ = true

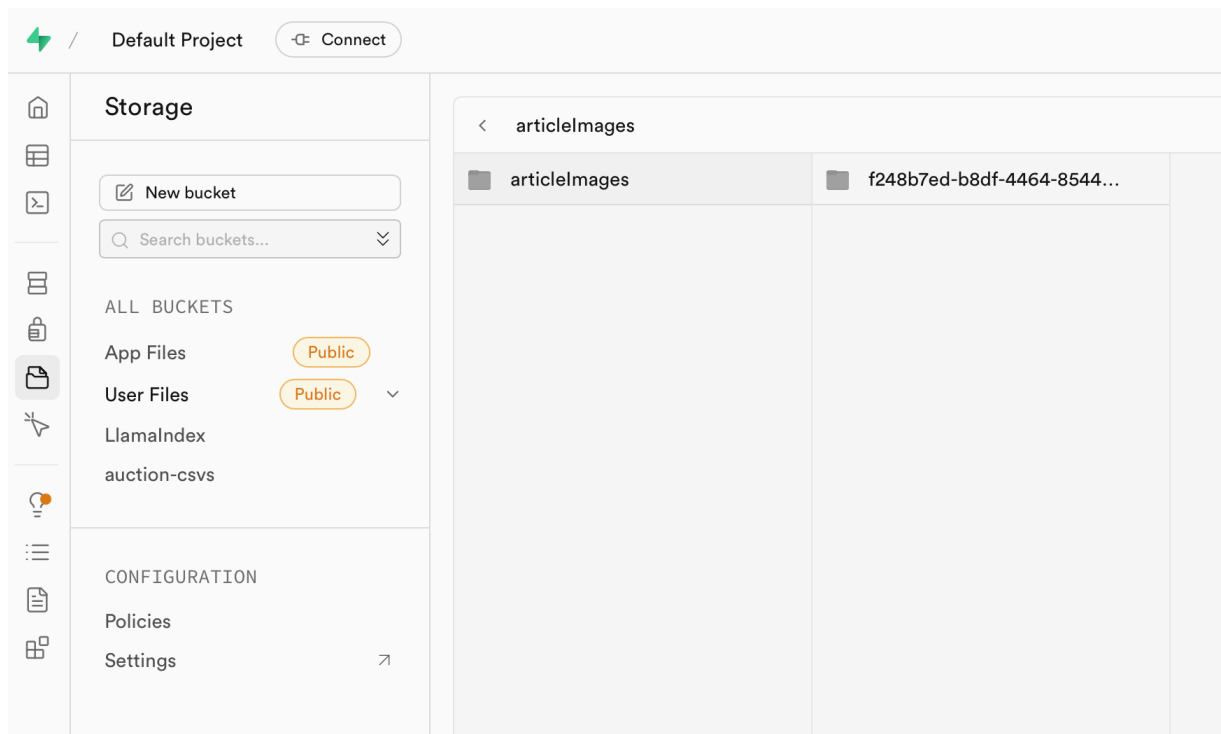
With the image generated, the system uploads this image to Supabase storage bucket “User Files” inside folder “articleImages” and inside a user’s own folder named with the user’s supabase auth id.

The system then retrieves the url of the uploaded image.

B) Load from local machine

The user can select an image file from their local

computer. The system uploads this image to Supabase storage bucket “User Files” inside folder “articleImages” and inside a user’s own folder named with the user’s supabase auth id. The system then retrieves the url of the uploaded image.



C) Loading a stock Image: The user can select sources from Pexels or Unsplash. The user will see a gallery with images and their authors/name are displayed in the screen. The user can select one of them and load to it supabase. The system uploads this image to Supabase storage bucket “User Files” inside folder “articleImages” and inside a user’s own folder named with the user’s supabase auth id. The system then retrieves the url of the uploaded image.

Api keys can be found in table api_keys. Search for provider unsplash or pexels and you will find the corresponding key_value

D) Generating an Infographic: To generate an infographic, similar the user will select a template from a dropdown list. It will show a sample image so the user can understand the look. Once the user selects the right template, if the user had previously selected a paragraph of text from the article body, the text will be shown in an editable text box (if no text was selected the text box will be blank and the user will have the option to describe what they want to fill the template with or a story that the template should tell). If there is text in the editable text box then a button is enabled to send the text along with a prompt to the default LLM that will read the text and create a JSON object with the format required by the template.

TECHNICAL DETAILS:

Following is code from my prototype in Noodl that you can use as a reference. You do not need to copy this code but use it to understand what I was doing.

1) LOAD STOCK IMAGE”

1.1 Unsplash:

```
async function queryUnsplash() {  
  const pageNumber = Inputs.PageNumber || 1;  
  const key = Inputs.key;  
  
  // Use optional chaining for safer access to nested properties  
  let prompt = Inputs.prompt  
  
  const url = `https://api.unsplash.com/search/photos?`
```

```

client_id=${key}&query=${(prompt)}&page=${pageNumber}
&per_page=10`;
const method = "GET";

try {
  const response = await fetch(url, { method });

  if (!response.ok) {
    throw new Error(`Unsplash API error! status: ${
{response.status}}`);
  }

  const data = await response.json();
  const totalPages = data.total_pages;
  Outputs.totalPages=totalPages;
  const firstTenImages = data.results.slice(0, 10).map(image
=> {
    const authorName = `${image.user?.first_name ?? ""} ${
{image.user?.last_name ?? ""}}`;
    return {
      id: image.id,
      url: image.urls.small,
      urlRaw: image.urls.raw,
      urlThumb: image.urls.thumb,
      urlSmall: image.urls.small,
      urlFull: image.urls.full,
      urlRegular: image.urls.regular,
      userId: image.user?.id, // Optional chaining
      userFirstName: image.user?.first_name, // Optional
chaining
      userLastName: image.user?.last_name, // Optional
chaining
      downloadUrl: image.links?.download, // Optional chaining
      downloadLocation: image.links?.download_location, //
Optional chaining
      author: `Unsplash Image - author: ${authorName.trim()}`,
      description: image.description || "",
    };
  });

  Outputs.firstTenImages = firstTenImages;
  Outputs.data = data;
  Outputs.Success();
  console.log("Success:", firstTenImages);

} catch (error) {
  Outputs.error = error.message; // Only send the error

```

```

message, not the entire object
    Outputs.Failure();
    console.error("queryUnsplash Error:", error);
  }
}
queryUnsplash()

```

1.1 Pexels:

```

async function queryPexels() {
  const key = Inputs.key;
  const perPage = Inputs.perPage || 10;
  const prompt = Inputs.prompt;
  const pexelsPage = Inputs.pexelsPage || "";
  let url = ""

  // Construct the URL properly
  if (pexelsPage === "Pexels Search") {
    url = `https://api.pexels.com/v1/search?query=${prompt}
    &page=2&per_page=${perPage}`;
  } else if (pexelsPage === "Pexels Curated") {
    url = `https://api.pexels.com/v1/curated?per_page=${perPage}
    `;
  } else {
    url = pexelsPage
  };
  const method = "GET";
  const headers = {
    Authorization: key
  };

  try {
    const response = await fetch(url, { method, headers });

    if (!response.ok) {
      throw new Error(`Pexels API error! status: ${
        {response.status}}`);
    }

    const data = await response.json();
    const next_page = data.next_page;
    const prev_page = data.prev_page;
    const totalPages = data.total_results/perPage ;
    const page = data.page;
  }
}

```



```
Outputs.totalPages = totalPages;
Outputs.next_page = next_page;
Outputs.prev_page = prev_page;
Outputs.curr_page = page;
```

```
    // Map the first ten images
    const firstTenImages = data.photos.slice(0,
perPage).map(image => ({
        id: image.id,
        width: image.width,
        height: image.height,
        urlPexels: image.url,
        author: "Pexels image by author " + image.photographer,
        photographer_url: image.photographer_url,
        photographer_id: image.photographer_id,
        avg_color: image.avg_color,
        original: image.src.original,
        large2x: image.src.large2x,
        large: image.src.large,
        url: image.src.medium,
        small: image.src.small,
        portrait: image.src.portrait,
        landscape: image.src.landscape,
        tiny: image.src.tiny,
        description: image.alt
    }));
```

```
Outputs.firstTenImages = firstTenImages;
Outputs.data = data;
Outputs.Success();
console.log("Success:", firstTenImages);
```

```
    } catch (error) {
        Outputs.error = error.message; // Only send the error
message, not the entire object
        Outputs.Failure();
        console.error("queryPexels Error:", error);
    }
}
```

```
queryPexels();
```

2) Generate AI image

```
const SDapiKey = Inputs.SDapiKey;
const fluxApiKey = Inputs.fluxApiKey;
const prompt = Inputs.prompt;
const image = Inputs.image || "";
const geminiAPIKey = Inputs.geminiAPIKey;
const strength = Inputs.strength / 100;
const aspectRatio = Inputs.aspectRatio;
const negativePrompt = Inputs.negativePrompt || "";
const imageToImage = Inputs.imageToImage;
const imagenAPIKey = Inputs.ImagenAPIKey;
const imageEngine = Inputs.imageEngine;
const SD_Model = Inputs.SD_Model;
const fluxModel = Inputs.fluxModel;
const imagenModel = Inputs.imagenModel;
const SD_EditMode = Inputs.SD_EditMode;
let url = ""
```

```
Outputs.status = formatCurrentDate() + ": Image Generation  
Started";
console.log("Image generation request started. Engine: " +  
imageEngine, ", SD_EditMode = ",SD_EditMode );
```

```
// GENERATE IMAGE
if (imageEngine === "Stable Diffusion") {
    generateImageWithStableDiffusion(SDapiKey, prompt,  
imageToImage, image, strength, aspectRatio, negativePrompt,  
SD_EditMode, SD_Model)
        .then(({ imageUrl, blob, file }) => {  
            setOutputs(imageUrl, blob, file, SD_Model);  
        })  
        .catch(handleError);
} else if (imageEngine === "Flux Pro") {
    generateImageWithFlux(fluxApiKey, prompt, fluxModel)
        .then(({ imageUrl, blob, file }) => {  
            setOutputs(imageUrl, blob, file, fluxModel);  
        })  
        .catch(handleError);
} else if (imageEngine === "Google Imagen") {
    generateImageWithGoogleImagen(prompt, imagenModel, geminiAPIKey)
```

```

        .then(({ imageUrl, blob, file }) => {
            setOutputs(imageUrl, blob, file, imagenModel);
        })
        .catch(handleError);
    } else {
        Outputs.error = "Invalid image engine selected";
    }
}

```

```

function setOutputs(imageUrl, blob, file, author) {
    Outputs.imageUrl = imageUrl;
    Outputs.blob = blob;
    Outputs.file = file;
    Outputs.author = author;
    Outputs.Success();
}

```

```

function handleError(error) {
    console.error('Error:', error);
    Outputs.error = error.message;
}

```

```

async function generateImageWithGoogleImagen(prompt,
imagenModel, geminiApiKey) {
    // No more JWT service needed!
    const url = `https://generativelanguage.googleapis.com/
v1beta/models/${imagenModel}:generateImages`;

    const body = JSON.stringify({
        prompt: prompt,
        config: {
            number_of_images: 1,
            aspect_ratio: "1:1" // or other ratios
        }
    });
}

```

```

    Outputs.status = formatCurrentDate() + ": Request Sent to
Google Imagen 3 - Waiting for Image Generation";

```

```

    try {
        const response = await fetch(url, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${geminiApiKey}`, //
Simple API key
                'Content-Type': 'application/json'
            },
            body: body
        });
    }

```

```

    });

    if (!response.ok) {
        const errorData = await response.json().catch(() =>
({}));
        throw new Error(`Google Imagen failed: $
{response.status} ${errorData.error?.message ||
response.statusText}`);
    }

    const responseData = await response.json();

    if (responseData.generated_images &&
responseData.generated_images.length > 0) {
        const imageBytes =
responseData.generated_images[0].image.image_bytes;

        // Convert base64 to Blob (if needed)
        const byteCharacters = atob(imageBytes);
        const byteNumbers = new
Array(byteCharacters.length);
        for (let i = 0; i < byteCharacters.length; i++) {
            byteNumbers[i] = byteCharacters.charCodeAt(i);
        }
        const byteArray = new Uint8Array(byteNumbers);
        const blob = new Blob([byteArray], { type: 'image/
png' });

        const file = new File([blob], 'generated_image.png',
{ type: 'image/png' });
        const imageUrl = URL.createObjectURL(blob);

        Outputs.status = formatCurrentDate() + ": Image
generation complete";
        return { imageUrl, blob, file };
    } else {
        throw new Error('No image generated in the
response');
    }
} catch (error) {
    console.error('Error:', error);
    Outputs.status = formatCurrentDate() + ": Error
generating image: " + error.message;
    throw error;
}
}

```

```
// FUNCTION TO GENERATE IMAGE WITH STABLE DIFFUSION
function generateImageWithStableDiffusion(apiKey, prompt,
imageToImage, image, strength, aspectRatio, negativePrompt,
SD_EditMode, SD_Model) {
    return makeStableDiffusionApiCall(apiKey, prompt,
imageToImage, image, strength, aspectRatio,
negativePrompt, SD_EditMode, SD_Model)
        .then(blob => {
            console.log("Stable Diffusion response received");
            Outputs.status = formatDate() + ": Image
generated successfully";
            const imageUrl = URL.createObjectURL(blob);
            const file = new File([blob],
'StableDiffusion.jpeg', { type: 'image/jpeg' });
            return { imageUrl, blob, file };
        });
}
```

```
// FUNCTION TO MAKE STABLE DIFFUSION API CALL
async function makeStableDiffusionApiCall(apiKey, prompt,
imageToImage, image, strength, aspectRatio, negativePrompt,
SD_EditMode = "Standard", SD_Model = 'SD3.5 Medium') {
    const formData = new FormData();
    console.log(`SD request being created. imageToImage = $
{imageToImage}, Strength: ${strength}, model: ${SD_Model}`);
```

```
    // Append required fields
    formData.append('prompt', prompt);
    formData.append('output_format', 'jpeg');
    formData.append('model', SD_Model);
    formData.append(
        'negative_prompt',
        '"blurry", "pixelated", "bad anatomy", "poor lighting",
"dullness", "ugly", "boring", "bad hands", "extra limbs",
"disfigured body", ${negativePrompt}`
    );
```

```
    let url; // Declare URL variable
```

```
    // Handle Edit-InPaint mode
    if (SD_EditMode === "Edit-InPaint") {
        url = 'https://api.stability.ai/v2beta/stable-image/
edit/inpaint';
```

```

        console.log("SD_EditMode = ", SD_EditMode, ", url: ",
url)
        if (image) {
            formData.append('image', image);
        }
    } else { // Handle Standard or Image-to-Image mode
        url = 'https://api.stability.ai/v2beta/stable-image/
generate/sd3';
        console.log("SD_EditMode = ", SD_EditMode, ", url: ",
url)
        if (imageToImage && image) {
            console.log("Processing Image-to-Image with image:",
image);
            Outputs.status = `${formatCurrentDate()}: Image-to-
Image Started`;
            formData.append('image', image);
            formData.append('mode', 'image-to-image');
            formData.append('strength', strength);
        } else {
            formData.append('aspect_ratio', aspectRatio);
        }
    }
}

```

```

// Log FormData contents for debugging
console.log("Form Data:");
for (const [key, value] of formData.entries()) {
    console.log(`${key}: ${value}`);
}

```

```

Outputs.status = `${formatCurrentDate()}: Request Sent -
Waiting for Image Generation`;

```

```

// Make the API call
return fetch(url, {
    method: 'POST',
    headers: {
        'Authorization': `Bearer ${apiKey}`,
        'Accept': 'image/*'
    },
    body: formData
})
    .then(response => {
        if (!response.ok) {
            Outputs.status = `${formatCurrentDate()}: Error
generating Image`;
            return response.json().then(errorData => {
                console.error('Error ID:', errorData.id);
            });
        }
    });

```

```

        console.error('Error Name:',
errorData.name);
        console.error('Error Details:',
errorData.errors);
        throw new Error(`Image generation failed: $
{errorData.message}`);
    });
    }
    return response.blob(); // Return the generated
image as a Blob
    })
    .catch(error => {
        console.error("Error during API call:", error);
        Outputs.status = `${formatCurrentDate()}: API Call
Failed`;
        throw error;
    });
}

```

```

// FUNCTION TO GENERATE IMAGE WITH FLUX (Polling version)
async function generateImageWithFlux(apiKey, prompt, fluxModel =
Inputs.fluxModel) {
    const generateUrl = 'https://api.fluxapi.ai/api/v1/flux/
kontext/generate';

```

```

    const body = JSON.stringify({
        prompt: prompt,
        enableTranslation: true,
        aspectRatio: Inputs.aspectRatio,
        outputFormat: 'jpeg',
        promptUpsampling: false,
        model: fluxModel
    });

```

```

    Outputs.status = `${formatCurrentDate()}: Request sent to
Flux API – waiting for image generation`;

```

```

    try {
        /* 1. Create the task
        ----- */
        const postResp = await fetch(generateUrl, {
            method: 'POST',
            headers: {
                'Authorization': `Bearer ${apiKey}`,
                'Content-Type': 'application/json'
            },
            body

```

```

    });

    if (!postResp.ok) {
        const e = await postResp.json().catch(() => ({}));
        throw new Error(`Flux job start failed: ${
            {postResp.status} ${e.msg || e.detail || postResp.statusText}
            TaskRecord: ${e}`);
    }

    const initialData = await postResp.json();

    // Extract taskId from the initial response (assuming
    // it's under 'data')
    const taskId = initialData?.data?.taskId ||
        initialData?.taskId || initialData?.result?.taskId;
    if (!taskId) {
        console.log("Initial Flux Response:", initialData);
        throw new Error('Flux job started, but the response
            did not contain a taskId. Check console for "Initial Flux
            Response."');
    }

    console.log('Flux taskId:', taskId);

    // *** FIX: Use the confirmed Polling URL (record-info)
    ***
    const pollingUrl = `https://api.fluxapi.ai/api/v1/flux/
        kontext/record-info?taskId=${taskId}`;
    console.log('Flux Polling URL:', pollingUrl);

    /* 2. Poll until finished
    ----- */
    const poll = () => {
        fetch(pollingUrl, {
            headers: { 'Authorization': `Bearer ${apiKey}` }
        }).then(r => {
            if (!r.ok) throw new Error('Poll request failed
                ' + r.status);
            return r.json();
        });
    };

    let pollData;
    const maxAttempts = 150; // 5 minutes max timeout (150
        attempts * 2s)
    for (let attempts = 0; attempts < maxAttempts; attempts+
        +) {

```



```
await new Promise(r => setTimeout(r, 2000)); // 2-second pause
```

```
pollData = await poll();  
const successFlag = pollData.data?.successFlag;  
  
console.log(`Poll Status Flag: ${successFlag}`);
```

```
    // Success status: 1 = SUCCESS  
    if (successFlag === 1) break;  
    console.log(pollData)  
    // Failure statuses: 2 = CREATE_TASK_FAILED, 3 =  
GENERATE_FAILED  
    if (successFlag === 2 || successFlag === 3) {  
        throw new Error(`Flux generation failed (Error  
Code ${successFlag}): ${pollData.data?.errorMessage || 'Check  
task record for details.'}, Poll data: ${pollData}`);  
    }
```

```
    Outputs.status = `${formatCurrentDate()}: Still  
waiting for image generation... Status: GENERATING`;  
}
```

```
    if (pollData?.data?.successFlag !== 1) {  
        throw new Error('Image generation timed out after 5  
minutes.');
```

```
    /* 3. Download the picture  
----- */  
    // Extract the result from the confirmed structure:  
data.response.resultImageUrl
```

```
    const imageUrl =  
pollData.data?.response?.resultImageUrl;  
  
    if (!imageUrl) throw new Error('No resultImageUrl in  
completed task response.');
```

```
    const imgResp = await fetch(imageUrl);  
    if (!imgResp.ok) throw new Error('Could not download  
finished image');
```

```
    const blob = await imgResp.blob();  
    const file = new File([blob], 'Flux.jpeg', { type:  
'image/jpeg' });
```

```
    Outputs.status = `${formatCurrentDate()}: Image
```

```
generation complete`;
    return { imageUrl, blob, file };
}
```

```
    } catch (error) {
        Outputs.status = `${formatCurrentDate()}: Error
generating Image: ${error.message}`;
        console.error('Error:', error);
        throw error;
    }
}
```

```
// FORMAT CURRENT Date
function formatCurrentDate() {
    const currentDate = new Date();

    // Array of month abbreviations
    const months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];

    // Get date components
    const month = months[currentDate.getMonth()];
    const day = String(currentDate.getDate()).padStart(2, '0');
    const year = currentDate.getFullYear();

    // Get time components
    let hours = currentDate.getHours();
    const minutes = String(currentDate.getMinutes()).padStart(2,
'0');

    // Convert to 12-hour format
    const ampm = hours >= 12 ? 'PM' : 'AM';
    hours = hours % 12;
    hours = hours ? hours : 12; // the hour '0' should be '12'
    hours = String(hours).padStart(2, '0');

    // Get timezone abbreviation
    const timeZone =
Intl.DateTimeFormat().resolvedOptions().timeZone;
    const timeZoneAbbr = new Intl.DateTimeFormat('en',
{ timeZoneName: 'short' })
        .formatToParts(currentDate)
        .find(part => part.type === 'timeZoneName').value;

    // Construct the formatted string
    return `${month}${day}${year}. ${hours}:${minutes} $
{timeZoneAbbr}`;
}
```

```
// CALL JWT Service in CLOUD RUN to request Google Token for
// AUTH
// It used the default Google service account in cloud run
async function callJwtService() {
  const url = 'https://jwt-token-service-445556218231.us-
east4.run.app/getToken'; // Replace with your Cloud Run URL
```

```
  try {
    const response = await fetch(url, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
    });
```

```
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${
        response.status}`);
    }
  }
```

```
  const data = await response.json();
  return data.token; // Adjust based on your API response
} catch (error) {
  console.error('Error calling JWT service:', error);
  throw error; // Rethrow or handle as needed
}
```

3) Generate Infograph

3.1) Find available Infograph templates:

```
async function fetchInfographicImage(Value) {
  try {
    let { data: infographic, error } = await
    Noodl.Variables.supabase
      .from('infographic')
      .select("*")
      .eq('id', Value);
```

```

    console.log("Supabase response:", { infographic, error });

    if (error) {
        throw new Error(`Error fetching infographic: ${
error.message}`);
    }

    if (!infographic || !Array.isArray(infographic) ||
infographic.length === 0) {
        console.log("No infographic data found");
        Outputs.Items = [];
        Outputs.Success();
        return;
    }

    console.log("Fetched infographic:", infographic);

    const image = infographic[0].sampleImage; // Correctly
accessing sampleImage

    console.log("Sample infographic:", image); // Changed from
sampleImage to image
    Outputs.image = image; // Ensure Outputs is set up to handle
this
    Outputs.Success();
} catch (err) {
    console.error("Error in fetchInfographic:", err);
    Outputs.Failure();
}
}

// Call the function
fetchInfographicImage(Inputs.Value);

```

This is a sample infographic record from table infographic:

```

{
  "created_at": "2024-10-07T22:54:30.69902+00:00",
  "CSS": "@import url('https://fonts.googleapis.com/css2?
family=Merriweather:ital@1&family=Oswald:wght@500&di
splay=swap');\n@import url('https://pro.fontawesome.com/
releases/v6.0.0-beta1/css/all.css');\n\n/* Reset and base

```

```

styles *\An*, *::before, *::after { \n margin: 0; \n padding: 0;
\n box-sizing: border-box;\n}\n\nbody {\n margin: 0;\n
padding: 0;\n display: flex;\n flex-direction: column;\n
align-items: center;\n justify-content: flex-start;\n width:
100%;\n min-height: 100vh;\n background-color:
#f5f5f5;\n font-family: 'Merriweather', serif;\n}\n\n.content-
wrapper {\n width: 100%;\n max-width: 60rem;\n margin:
0 auto;\n padding: 1rem 1rem 0;\n}\n\n/* Heading styles */
\nh1 {\n font-size: 2.0rem;\n font-family: 'Oswald', sans-
serif;\n text-align: center;\n font-weight: normal;\n
margin-bottom: 1rem;\n}\n\n/* Ordered list styles *\nol { \n
display: grid;\n grid-template-columns: repeat(auto-fit,
minmax(12rem, 1fr));\n gap: 1rem;\n width: 100%;\n
padding: 1rem 1rem 0;\n margin-bottom: 0;\n list-style:
none;\n counter-reset: count;\n}\n\n/* Card styles *\nol > li
{\n --card-background-color: #f0f0f0;\n --card-text-color:
#0F0F0F;\n --card-border-radius: 0.5rem;\n --card-
padding-block: 1.5rem;\n --card-padding-inline: 1rem;\n --
outset-size: 0.75rem;\n --outset-background-color:
#e5e5e5;\n --number-font-size: 3rem;\n --number-
overlap: 0.5rem;\n --number-font-family: 'Oswald', sans-
serif;\n --number-font-weight: 500;\n \n margin: var(--
outset-size);\n margin-top: calc(var(--number-font-size) -
var(--number-overlap));\n border-radius: var(--card-
border-radius);\n padding: var(--card-padding-block) var(--
card-padding-inline);\n color: var(--card-text-color);\n
background-color: var(--card-background-color);\n box-
shadow: \n    inset 1px 1px 0px rgb(255 255 255 / .5),\n
inset -1px -1px 0px rgb(0 0 0 / .25),\n    calc(var(--outset-
size) * .25) calc(var(--outset-size) * .25) calc(var(--outset-

```

```

size) * .5) rgb(0 0 0 / .25);\n position: relative;\n counter-
increment: count;\n}\n\nol > li::after {\n content:
counter(count, decimal-leading-zero);\n position:
absolute;\n bottom: calc(100% - var(--number-overlap));\n
left: 50%;\n transform: translateX(-50%);\n color: var(--
accent-color);\n font-family: var(--number-font-family);\n
font-weight: var(--number-font-weight);\n font-size: var(--
number-font-size);\n line-height: 1;\n z-index: -1;\n}\n\nol
> li::before {\n content: \"\";\n position: absolute;\n width:
calc(100% + (var(--outset-size) * 2));\n height: 100%;\n
bottom: calc(var(--outset-size) * -1);\n left: calc(var(--
outset-size) * -1);\n z-index: -1;\n border-bottom-left-
radius: calc(var(--card-border-radius) + var(--outset-size));
\n border-bottom-right-radius: calc(var(--card-border-
radius) + var(--outset-size));\n background-color: var(--
outset-background-color);\n background-image: \n
linear-gradient(to left, var(--outset-background-color)
calc(var(--outset-size) * 2), transparent 0),\n linear-
gradient(135deg, var(--accent-color) 80%, var(--outset-
background-color) 0); \n}\n\n/* Card content styles */
\n.icon {\n font-size: 2rem;\n text-align: center;\n margin-
bottom: calc(var(--card-padding-block) * .5);\n}\n\n.title {\n
text-transform: uppercase;\n font-family: 'Oswald', sans-
serif;\n text-align: center;\n color: var(--accent-color);\n}
\n\n.descr {\n color: var(--text-color);\n font-size: 0.75rem;
\n text-align: center;\n}",
  "HTML": "<div class=\"content-wrapper\">\n
<h1>+title+</h1>\n <ol>\n   <li style=\"--accent-color:
#FF6F00\">\n     <div class=\"icon\"><i class=\"fa-light fa-
+icon1+\"></i></div>\n     <div class=\"title\">+item1+</

```

```

div>\n    <div class=\"descr\">+description1+</div>\n  </li>\n  <li style=\"--accent-color: #008DC2\">\n    <div class=\"icon\"><i class=\"fa-light fa-+icon2+\"></i></div>\n    <div class=\"title\">+item2+</div>\n    <div class=\"descr\">+description2+</div>\n  </li>\n  <li style=\"--accent-color: #0B456A\">\n    <div class=\"icon\"><i class=\"fa-light fa-+icon3+\"></i></div>\n    <div class=\"title\">+item3+</div>\n    <div class=\"descr\">+description3+</div>\n  </li>\n  <li style=\"--accent-color: #6A829A\">\n    <div class=\"icon\"><i class=\"fa-light fa-+icon4+\"></i></div>\n    <div class=\"title\">+item4+</div>\n    <div class=\"descr\">+description4+</div>\n  </li>\n</ol>\n</div>\",

```

"numberOfItems": 4,

"JS": null,

"prompt": "You will be writing the content for a simple infographics that shows a title and then a list of four items with their short descriptions. Items are named item1, item2, item3, item4 and their respective descriptions description1, description2, description3, description4.. Title can have a maximum of 20 characters, items a max of 13 characters and description a maximum of 30 characters. You also need to provide Font Awesome icon names (e.a. ring or house) in fields icon1, icon2, icon3 and icon4 that best relate to the topic in the corresponding item.\nOutput format: The output Must be in the following JSON

```

Format\n\n{\n  \"title\": \"\", \n  \"items\": [\n    {\n      \"item1\": \"\", \n      \"description1\": \"\", \n      \"icon1\": \"\", \n    }, \n    {\n      \"item2\": \"\", \n      \"description2\": \"\",

```

```

\n  \"icon2\": \"\\\"\\n  },\n  {\n    \"item3\": \"\\\",
\"description3\": \"\\\",
    \"icon3\": \"\\\"\\n  },\n  {\n
\"item4\": \"\\\",
    \"description4\": \"\\\",
    \"icon4\":
\"\\\"\\n  }\n ],\n  \"summary\": \"\\\"\\n}\\n\",
    \"jsonStructure\": \"{\\n  \"title\": \"\\\",
    \"items\": [\n  {\n
\"item1\": \"\\\",
    \"description1\": \"\\\"\\n  },\n  {\n
\"item2\": \"\\\",
    \"description2\": \"\\\"\\n  },\n  {\n
\"item3\": \"\\\",
    \"description3\": \"\\\"\\n  },\n  {\n
\"item4\": \"\\\",
    \"description4\": \"\\\"\\n  }\n ],\n
\"summary\": \"\\\"\\n}\\n\",
    \"type\": null,
    \"height\": 600,
    \"width\": 900,
    \"clipX\": 5,
    \"clipY\": 0,
    \"clipWidth\": 900,
    \"clipHeight\": 350,
    \"sampleImage\": \"https://sbcontent.aichieve.net/storage/
v1/object/public/App%20Files/Infographics/
file_1733231112671_919.png\",
    \"Value\": \"2\",
    \"Label\": \"4 OL Cards\"
  }

```

3.2 Get CSS/HTML Template details:

```

const id = Inputs.infographId;
let items = Inputs.items;
console.log('items:', items)

```

```

// Check if items is an array

```



```
let infographics = items.find(item => String(item.Value) ===
String(id));
console.log('items:', items)
console.log('infographics:', infographics)
```

```
// Check if an item was found
```

```
if (infographics) {
  Outputs.CSS = infographics.CSS || '';
  Outputs.clipX = infographics.clipX || '';
  Outputs.clipY = infographics.clipY || '';
  Outputs.clipWidth = infographics.clipWidth || '';
  Outputs.clipHeight = infographics.clipHeight || '';
  Outputs.HTML = infographics.HTML || '';
  Outputs.height = infographics.height || '';
  Outputs.width = infographics.width || '';
  Outputs.prompt = infographics.prompt || '';
  Outputs.jsonStructure = infographics.jsonStructure || '';
  Outputs.infographicsName = infographics.Label || ''
} else {
  console.warn(`No item found with id: ${id}`);
  Outputs.CSS = '';
  Outputs.HTML = '';
}
```

```
console.log("infographicId 1:", id)
```

```
// Define the async function
```

```
async function fetchInfographicDetails(infographicId) {
  try {
    console.log("infographicId 2:", infographicId)
    // Fetch data from Supabase
    let { data: infographicDetails, error } = await
Noodl.Variables.supabase
      .from('infographicDetails')
      .select("*")
      .eq('infographicId', infographicId);
```

```
    // Log the raw response for debugging
```

```
    console.log("Supabase response infographicDetails:",
{ infographicDetails, error });
```

```
    // Check for errors in fetching data
```

```
    if (error) {
      throw new Error(`Error fetching infographicDetails: $
{error.message}`);
    }
  }
}
```

```

    // Check if infographic data exists and is an array
    if (!infographicDetails || !
Array.isArray(infographicDetails) || infographicDetails.length
=== 0) {
        console.log("No infographicDetails data found");
        Outputs.infographicDetails = infographicDetails;
        Outputs.Success();
        return;
    }

    // Log the infographic data
    console.log("Fetched infographic:", infographicDetails);

    // Set outputs
    Outputs.infographicDetails = infographicDetails;

    Outputs.Success();
} catch (err) {
    console.error("Error in fetchInfographic:", err);
    Outputs.Failure(err.message);
}
}

// Call the function correctly and handle the Promise
fetchInfographicDetails(id).then(() => {
    console.log("Fetch completed");
}).catch((error) => {
    console.error("Error in main execution:", error);
});

```

3.3 LLM call to generate a JSON file from an input text in an output format required by the infographic template:

```

const supabase = await Noodl.Variables.supabase;
const perplexityKey = Inputs.perplexitykey;
const geminiKey = Inputs.geminiKey;
let promptText = Inputs.promptText;
const openAIKey = Inputs.openAIKey;
const LLM = Inputs.LLM || "Gemini";
const infographic = Inputs.infograpic;
const topic = Inputs.topic;
promptText = "You need to find something related to the topic
that can be shown in an infographic and then fill the

```

corresponding fields in the output JSON file. Following is the topic: ' + topic + "'. " + promptText;

```
async function processLLMResponse() {
```

```
  try {  
    let extractedText;
```

```
    // CALL LLM Gemini or Perplexity
```

```
    console.log("CALL LLM to create Infographic")
```

```
    if (LLM === "Perplexity") {
```

```
      console.log("LLM CALL 2 Perplexity");
```

```
      extractedText = await callPerplexityAPI(promptText,
```

```
perplexityKey);
```

```
    } else if (LLM === "Gemini") {
```

```
      console.log("LLM CALL 2 GEMINI");
```

```
      console.log("promptText:", promptText);
```

```
      extractedText = await callGeminiAPI(promptText, geminiKey,
```

```
true);
```

```
    } else if (LLM === "OpenAI") {
```

```
      console.log("LLM CALL 2 Chat Gpt 4o");
```

```
      extractedText = await callOpenAIChatGPT(promptText,
```

```
openAIKey, useJsonOutput = true)
```

```
    } else {
```

```
      throw new Error("Invalid LLM specified");
```

```
    }
```

```
    console.log("Outline Generation Completed. ORIGINAL TEXT  
BEFORE JSON extraction: >> ", extractedText )
```

```
    extractedText = extractJson(extractedText)
```

```
    console.log("Outline Generation Completed. TEXT AFTER JSON  
extraction: >> ", extractedText)
```

```
    // Check if the output is Parseable
```

```
    console.log("Check if LLM output is Parseable")
```

```
    let isParseable = isJSONParseable(extractedText);
```

```
    if (isParseable) {
```

```
      Outputs.Answer = extractedText;
```

```
      Outputs.status = "Outline Generation Completed"
```

```
      Outputs.Success();
```

```
    } else {
```

```
      // Second attempt: Use LLM to fix JSON
```

```
      console.log("Second attempt: Use LLM to fix JSON")
```

```
      const fixPrompt = "The following JSON text is not  
parseable. Please fix it and output the correct text that is  
ready for parsing. Your response Text will be directly sent to a  
JSON.parse(responseText) so do not include any other extra text:  
" + extractedText;
```

```
      if (LLM === "Perplexity") {
```

```
        extractedText = await callPerplexityAPI(fixPrompt,
```

```

perplexityKey);
    } else {
        extractedText = await callGeminiAPI(fixPrompt,
geminiKey, false);
    }
    // Apply SimpleCleaning again to the fixed JSON
    try {
        extractedText = await SimpleCleaning(extractedText);
    } catch (cleaningError) {
        console.warn("Simple cleaning failed after LLM fix:",
cleaningError.message);
    }
    Outputs.Answer = extractedText;
    console.log("Outline Generation Completed. Text after
cleaning:", extractedText)
    Outputs.status = "Outline Generation Completed"
    Outputs.Success();
}
} catch (error) {
    console.error("Error in processLLMResponse:", error);
    Outputs.error = error.message;
    Outputs.status = "Infographic Generation Failed. Try
generating again"
    Outputs.Failure();
}
}
}

```

```

function isJSONParseable(jsonString) {
    try {
        JSON.parse(jsonString);
        return true;
    } catch (error) {
        return false;
    }
}

```

```

async function callGeminiAPI(promptText, geminiKey,
useJsonOutput = true) {
    try {
        const generationConfig = {
            // temperature: 0.7,
            topK: 40,
            topP: 0.95,
            maxOutputTokens: 2048
        };
    }
}

```

```

    if (useJsonOutput) {
        generationConfig.response_mime_type = "application/json";
    }

```

```

    const response = await fetch("https://
    generativelanguage.googleapis.com/v1beta/models/gemini-2.5-
    flash:generateContent?key=" + geminiKey, {
        method: "POST",
        headers: {
            'Content-Type': "application/json"
        },
        body: JSON.stringify({
            contents: [{
                parts: [{
                    text: promptText
                }]
            }],
            generationConfig: generationConfig
        })
    });

```

```

    if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
    }

```

```

    const data = await response.json();
    return data.candidates[0].content.parts[0].text;
} catch (error) {
    console.error("Error calling Gemini API:", error);
    throw error;
}
}

```

```

async function callOpenAIChatGPT(promptText, openAIKey,
useJsonOutput = true) {
    try {
        const generationConfig = {
            // model: "gpt-4-1106-preview",
            // model: "gpt-4o"
            model: "gpt-4o-mini",
            temperature: 0.7,
            max_tokens: 2048,
            top_p: 0.95,
            frequency_penalty: 0,
            presence_penalty: 0

```

```
};
```

```
const headers = {  
  'Content-Type': 'application/json',  
  'Authorization': `Bearer ${openAIKey}`  
};
```

```
const messages = [  
  {  
    role: "system",  
    content: useJsonOutput ? "You must respond in JSON  
format. Your response should be a valid JSON object." : "You are  
a helpful assistant."  
  },  
  { role: "user", content: promptText }  
];
```

```
const requestBody = {  
  messages: messages,  
  ...generationConfig  
};
```

```
if (useJsonOutput) {  
  requestBody.response_format = { type: "json_object" };  
}
```

```
const response = await fetch("https://api.openai.com/v1/  
chat/completions", {  
  method: "POST",  
  headers: headers,  
  body: JSON.stringify(requestBody)  
});
```

```
if (!response.ok) {  
  throw new Error(`HTTP error! status: ${response.status}`);  
}
```

```
const data = await response.json();  
return data.choices[0].message.content;  
} catch (error) {  
  console.error("Error calling OpenAI API:", error);  
  throw error;  
}  
}
```

```
async function callPerplexityAPI(promptText, perplexityKey) {  
  try {
```

```

const response = await fetch("https://api.perplexity.ai/
chat/completions", {
  method: "POST",
  headers: {
    'authorization': `Bearer ${perplexityKey}`,
    'Content-Type': "application/json",
    'accept': "application/json"
  },
  body: JSON.stringify({
    "model": "llama-3.1-sonar-large-128k-online",
    "messages": [
      {
        "role": "system",
        "content": "Be precise"
      },
      {
        "role": "user",
        "content": promptText
      }
    ]
  })
});

```

```

if (!response.ok) {
  throw new Error(`HTTP error! status: ${response.status}`);
}

```

```

const data = await response.json();
return data.choices[0].message.content;
} catch (error) {
  console.error("Error calling Perplexity API:", error);
  throw error;
}

```

```

function extractJson(textResponse) {
  // Remove any non-printable characters
  textResponse = textResponse.replace(/[\x00-\x1F\x7F-\x9F]/g,
  "");

```

```

  // Replace curly quotes with straight quotes
  textResponse = textResponse.replace(/[\u201C\u201D]/g, '"');

```

```

  // Remove any leading or trailing whitespace
  textResponse = textResponse.trim();

```

```

  try {
    // Attempt to parse the entire string

```

```

    const jsonObj = JSON.parse(textResponse);
    // If successful, return the string
    return textResponse;
  } catch (error) {
    console.error("Error parsing JSON:", error);

    // If parsing fails, try to fix common issues
    let fixedText = textResponse
      .replace(/\n/g, "\\n") // Replace newlines with
escaped newlines
      .replace(/\r/g, "\\r") // Replace carriage returns
with escaped carriage returns
      .replace(/\t/g, "\\t") // Replace tabs with escaped
tabs
      .replace(/\\/g, "\\") // Escape backslashes
      .replace(/\u2028/g, "\\u2028") // Replace line
separator
      .replace(/\u2029/g, "\\u2029"); // Replace paragraph
separator

    try {
      // Try parsing again after fixes
      const jsonObj = JSON.parse(fixedText);
      return fixedText; // If successful, return the fixed
string
    } catch (error) {
      console.error("Error parsing JSON after fixes:",
error);
      return null;
    }
  }
}
// Call the main function
processLLMResponse();

```